

Отчёт

Rate Limiter

Выполнили:

Григорьев Тимофей M3100 (Алгоритмы)

Коткин Михаил M3100 (Benchmarks)

Коконов Никита M3100 (Unit-тесты)

Крылов Дмитрий M3103 (Отчёт)

Маслов Денис M3101 (Benchmarks)

Аннотация

Цель исследования — определить оптимальный алгоритм ограничения частоты запросов для различных сценариев нагрузки на основе метрик (пропускная способность, задержка, точность ограничения, использование памяти, обработка burst-трафика).

Задачи работы:

- Реализация потокобезопасного Rate Limiter, поддерживающего множество клиентов (ключей).
- Реализация трёх алгоритмов: Token Bucket, Leaking Bucket, Sliding Window Log.
- Реализация механизма TTL и фоновой/ленивой очистки старых ключей.
- Разработка нагрузочного тестирования с параметризуемыми паттернами трафика.
- Проведение экспериментальных замеров и анализ результатов.

1. Теоретическая часть

В современных высоконагруженных системах критически важно обеспечивать отказоустойчивость и стабильность работы серверов.

Rate Limiter (ограничитель частоты запросов) — это программный механизм, контролирующий интенсивность входящего трафика. Он позволяет задавать строгие квоты на количество операций, которые конкретный клиент может выполнить за определённый интервал времени.

Внедрение алгоритмов ограничения скорости решает сразу несколько ключевых задач: предотвращает исчерпание вычислительных ресурсов сервера, защищает инфраструктуру от вредоносных атак (таких как DDoS или перебор паролей) и позволяет справедливо распределять пропускную способность системы между всеми пользователями.

1.1. Алгоритмы ограничения частоты запросов

Token Bucket (Корзина токенов)

Алгоритм представляет собой «корзину», в которую с постоянной скоростью добавляются токены. При обработке каждого запроса один токен удаляется из корзины. Если токенов недостаточно, запрос отбрасывается. Ключевым отличием алгоритма является то, что токены могут накапливаться при простое системы (вплоть до максимальной вместимости корзины).

Математическая модель

Пусть b — максимальная вместимость корзины, а r — скорость

пополнения токенов (штук в секунду). Количество токенов $T(t)$ в момент времени t вычисляется как:

$$T(t) = \min(b, T(t_{\text{prev}}) + (t - t_{\text{prev}}) \cdot r)$$

Где t_{prev} — время предыдущего запроса.

Преимущества и недостатки

- **Плюсы:** Эффективен по использованию памяти. Главное преимущество — способность обрабатывать кратковременные всплески трафика (bursts), так как накопленные за время простоя токены позволяют мгновенно пропустить пачку запросов.
- **Минусы:** Если размер корзины (b) настроен неправильно, внезапный burst нагрузки может перегрузить целевой сервер, так как алгоритм пропустит эти запросы одновременно.

Сценарии применения

Широко применяется в **API Gateway** и для ограничения действий пользователя (например, публичные API-интерфейсы). Также существуют вариации с несколькими корзинами для приоритизации трафика на сетевых интерфейсах.

Leaking Bucket (Протекающее ведро)

Алгоритм обеспечивает интуитивно понятный подход с использованием FIFO-очереди. Поступающие запросы добавляются в конец очереди («ведра»). Через равные промежутки времени первый элемент извлекается и обрабатывается. Если очередь заполнена, новые поступающие запросы отбрасываются (или «утекают»).

Математическая модель

Пусть c — максимальный размер очереди, а r — константная скорость обработки (запросов в секунду). Заполненность очереди $Q(t)$ перед новым запросом:

$$Q(t) = \max(0, Q(t_{\text{prev}}) - (t - t_{\text{prev}}) \cdot r)$$

При поступлении запроса: если $Q(t) < c$, он помещается в очередь. В противном случае — отбрасывается.

Преимущества и недостатки

- **Плюсы:** Гарантированно сглаживает всплески (даже после простоя) и обрабатывает запросы с абсолютно одинаковой скоростью. Эффективен по памяти (размер очереди ограничен) и легко внедряется на одном балансировщике.

- **Минусы:** При резком увеличении трафика очередь забивается старыми запросами, лишая систему возможности быстро обработать свежие. Нет строгих гарантий по времени обработки (зависит от позиции в очереди). Сложно координировать глобальные лимиты между несколькими балансировщиками.

Сценарии применения

Оптимален для сглаживания трафика и асинхронных систем (например, очереди сообщений). Хорошо подходит для **защиты от DDoS** на уровне инфраструктуры, так как жестко лимитирует пропускную способность, предотвращая перегрузку бэкенда.

Sliding Window Log (Скользящий журнал)

Алгоритм предполагает точное отслеживание временных меток (timestamp) каждого запроса пользователя. Записи сохраняются в структуру данных (например, hash set) и сортируются. При каждом новом запросе система удаляет устаревшие метки, вышедшие за пределы скользящего окна, и вычисляет количество оставшихся записей. Если их число превышает лимит, запрос отклоняется.

Математическая модель

Пусть W — размер временного окна в секундах, а L — множество временных меток предыдущих запросов клиента. При поступлении нового запроса в момент t :

1. Множество очищается от старых запросов: $L_{\text{new}} = \{x \in L \mid x \geq t - W\}$
2. Если размер $|L_{\text{new}}| < \text{limit}$, запрос принимается, а метка t добавляется в L_{new} .

Преимущества и недостатки

- **Плюсы:** Идеальная точность ограничения. Алгоритм плавно отслеживает время каждого конкретного запроса, благодаря чему не создает искусственных «слепых зон» при переходе между временными интервалами. Поскольку трафик оценивается индивидуально для каждого клиента в любой момент времени, исключаются пиковые всплески нагрузки от одновременных действий множества пользователей.
- **Минусы:** Крайне высокая стоимость вычислений и памяти. Хранение timestamp каждого запроса требует много места. Кроме того, пересчет логa при каждом запросе плохо масштабируется на распределенных кластерах.

Сценарии применения

Используется для **ограничения действий пользователя**, где критически важна абсолютная точность (например, финансовые транзакции или строгие квоты платных API). Совершенно не подходит для **защиты от DDoS**, так как вычисление логов само по себе может исчерпать ресурсы сервера при масштабной атаке.

Сводное сравнение алгоритмов

Алгоритм	Преимущества	Недостатки	Использование памяти	Лучший сценарий
Token Bucket	Эффективен по памяти. Отлично обрабатывает кратковременные всплески (bursts).	При неправильной настройке размера корзины всплеск может перегрузить целевой сервер.	Низкое (хранятся токены и timestamp)	API Gateway, публичные квоты
Leaking Bucket	Гарантированно сглаживает всплески и обрабатывает запросы с абсолютно одинаковой скоростью.	Очередь забивается старыми запросами при резких всплесках, нет строгих гарантий по времени обработки.	Низкое (размер очереди ограничен)	Сглаживание трафика, очереди сообщений, защита от DDoS
Sliding Window Log	Идеальная точность лимитов. Плавно учитывает запросы во времени, исключая искусственные всплески нагрузки.	Крайне ресурсоемкий. Требует много памяти для хранения меток и CPU для их пересчета.	Высокое (хранится каждый лог запроса)	Финансовые API, строгие ограничения

1.2. Паттерны трафика

Для корректной оценки алгоритмов ограничения частоты запросов необходимо проанализировать их поведение в условиях различных профилей нагрузки. В рамках данного исследования рассматриваются пять ключевых паттернов трафика и их соответствие реальным сценариям эксплуатации систем.

Uniform (Равномерный трафик)

Характеризуется строго константной интенсивностью запросов без выраженных всплесков или падений.

Реальные сценарии: Межсерверное взаимодействие (M2M), непрерывный стриминг телеметрии с IoT-устройств или работа систем внутреннего мониторинга (например, регулярный опрос метрик сервером).

Burst (Пиковая нагрузка)

Представляет собой резкое и кратковременное многократное увеличение объема трафика на фоне низкой базовой нагрузки.

Реальные сценарии: Старт крупных онлайн-распродаж, массовая рассылка push-уведомлений клиентам или публикация срочных новостей, вызывающая лавинообразный приток пользователей.

Diurnal (Суточный цикл)

Плавное волнообразное изменение интенсивности трафика, привязанное к времени суток и коррелирующее с периодами бодрствования пользователей.

Реальные сценарии: Классические B2C-сервисы (социальные сети, стриминговые платформы), где пик нагрузки приходится на вечерние часы, а глубокий спад — на ночное время.

Sparse + Burst (Разреженный со всплесками)

Большую часть времени система находится в состоянии простоя (единичные запросы или их полное отсутствие), однако в строго определенные моменты генерируется пиковая нагрузка.

Реальные сценарии: Пакетная обработка данных (batch processing), запуск тяжелых вычислительных задач по расписанию (cron-скрипты, генерация отчетов в конце дня) или работа систем экстренного оповещения.

Zombie (Паразитный трафик)

Монотонный, непрерывный трафик, не подчиняющийся суточным циклам или бизнес-логике приложения. Часто генерируется в фоновом режиме и игнорирует ответы сервера.

Реальные сценарии: Активность вредоносных ботов, скрипты для автоматического парсинга (скрапинга) контента, «осиротевшие» фоновые процессы на устройствах клиентов или медленные DDoS-атаки прикладного уровня.

1.3. Параметры системы, задаваемые пользователем

Для проведения корректного нагрузочного тестирования и эмуляции реальных условий эксплуатации в системе предусмотрена гибкая настройка параметров. Ниже описаны ключевые конфигурационные величины, задаваемые перед началом каждого эксперимента.

Лимиты (Rate и Burst Capacity)

Базовые ограничения пропускной способности. Параметр rate определяет целевую скорость обработки (количество разрешенных

запросов в секунду). Параметр `burst_capacity` (применимо для `Token Bucket`) задает размер «окна толерантности» — максимальное количество запросов, которое система готова пропустить одновременно при наличии накопленной квоты.

Влияние: Данные параметры напрямую определяют строгость алгоритма и его способность сглаживать микро-всплески легитимного трафика.

TTL ключей (Time-to-Live)

Время жизни состояния неактивного клиента в памяти системы. Если от конкретного ключа не поступает запросов в течение заданного интервала TTL, его данные помечаются как устаревшие и удаляются механизмом фоновой очистки.

Влияние: Критически важный параметр для предотвращения исчерпания памяти при обработке бесконечного потока разовых посетителей или активности вредоносных ботов (паттерн `Zombie`).

Длительность тестирования

Временной интервал, в течение которого генератор нагрузки отправляет запросы в систему согласно выбранному паттерну.

Влияние: Длительность должна быть достаточной для выхода системы на стабильный режим работы. Это позволяет нивелировать погрешности от начального «прогрева» кэшей процессора и аллокатора памяти, а также дает время на срабатывание фоновых процессов очистки устаревших ключей.

Количество потоков-клиентов

Число параллельных рабочих потоков, генерирующих запросы от лица множества уникальных ключей.

Влияние: Задает уровень конкурентного доступа. Позволяет проверить эффективность шардирования, оценить накладные расходы на ожидание разблокировки мьютексов и доказать отсутствие состояний гонки данных при пиковых нагрузках.

Количество повторных запусков

Число итераций одного и того же эксперимента с последующим вычислением медианных и средних значений результатов.

Влияние: Гарантирует статистическую значимость исследования. Повторные запуски позволяют исключить влияние случайных факторов среды исполнения, таких как прерывания планировщика ОС или активность фоновых системных процессов.

1.4. Метрики оценки экспериментов

Для объективного сравнения реализованных алгоритмов в условиях различных профилей нагрузки используется следующий набор метрик:

Accuracy (Точность ограничения)

Измеряется как процентное отклонение реально пропущенных запросов от заданного лимита. Данная метрика критически важна для подтверждения корректности работы механизмов синхронизации. Она доказывает успешное решение проблемы «check-then-act» при конкурентном доступе множества потоков к счётчикам одного ключа и гарантирует отсутствие гонок данных.

Throughput (Пропускная способность)

Измеряется в количестве запросов в секунду (RPS), которые сам Rate Limiter способен обработать в чистом виде (без учета сетевых задержек). Характеризует процессорные накладные расходы алгоритма на захват мьютексов, работу с атомиками и хеш-таблицами.

p50 / p99 latency (Перцентили задержки)

Измеряется в микросекундах или наносекундах. Значение p50 показывает базовую скорость проверки лимита (когда нет жесткой конкуренции потоков). Значение p99 иллюстрирует задержку для 1% худших случаев — например, когда поток вынужден ожидать блокировки из-за срабатывания фоновой очистки или реаллокации памяти в структурах данных.

Memory usage (Использование памяти)

Измеряется в байтах на каждого уникального клиента (ключ). Отражает объем памяти, занимаемый внутренним состоянием: от двух базовых переменных в алгоритме Token Bucket до динамических структур для хранения временных меток в Sliding Window Log.

Burst handling (Обработка всплесков)

Оценивает поведение алгоритма при поступлении пиковой нагрузки. Фиксирует, какое количество запросов сверх стандартного лимита алгоритм пропускает единоразово (в пределах параметра burst_capacity), и насколько быстро он переходит в режим блокировки после исчерпания емкости.

Eviction count (Количество очисток)

Абсолютное число неактивных ключей, удаленных из хеш-таблицы механизмом ленивой очистки или фоновым потоком по истечении

параметра TTL. Метрика доказывает отсутствие утечек памяти и эффективность управления жизненным циклом клиентов.

1.5. Потокобезопасность и гонки данных

В высоконагруженных системах запросы от одного и того же клиента могут обрабатываться параллельно пулом рабочих потоков сервера. Это неизбежно приводит к конкурентному доступу к внутреннему состоянию алгоритма ограничения (счётчикам токенов, логам времени).

Проблема «Check-then-act» (Проверь, затем сделай)

Классическая уязвимость многопоточной среды. Возникает, когда поток успешно проверяет наличие квоты (check), но перед фактическим списанием токена (act) планировщик ОС прерывает его работу. В этот момент другой поток также проходит проверку и списывает ту же самую квоту. По возвращении первый поток завершает начатое действие, уводя счетчик в отрицательное значение. В результате система пропускает больше запросов, нарушая метрику Ассурасу.

Ограничения `std::atomic`

Хотя использование `std::atomic` защищает отдельные переменные от повреждения, в контексте алгоритмов Rate Limiter этого подхода зачастую недостаточно. Большинству алгоритмов требуется одновременное обновление сразу нескольких полей — например, и счетчика токенов, и метки времени. Гарантировать атомарность таких составных операций через сложные Compare-And-Swap циклы крайне ресурсоемко и ведет к запутанному коду.

Блокировки на уровне ключа (Mutex per key)

Надежный подход, при котором состояние каждого клиента защищается собственным примитивом синхронизации (например, `std::mutex`). Вся логика метода `Access` оборачивается в критическую секцию (`std::lock_guard`), что полностью исключает проблему «check-then-act». Основной недостаток — избыточное потребление памяти (один мьютекс на каждого клиента) и разрозненность объектов в куче, что ухудшает локальность данных.

Шардирование ключей (Sharded maps) и оптимизация памяти

Для баланса между потокобезопасностью и пропускной способностью применяется разбиение хранилища на фиксированное количество шардов. Каждый шард отвечает за свою группу ключей и защищается одним общим мьютексом. Запросы к разным шардам выполняются абсолютно параллельно.

Устранение False Sharing: При плотной упаковке шардов в памяти возникает аппаратная проблема «ложного разделения». Независимые потоки, обращаясь к разным мьютексам, могут инвалидировать кэш друг друга, если эти данные лежат в одной кэш-линии процессора. Для решения этой проблемы в высокопроизводительных Rate Limiter'ах применяется выравнивание памяти (placement new и кастомные аллокаторы), гарантирующее, что данные разных шардов аппаратно разнесены блоками по границам кэш-линий.

2. Архитектура и реализация программного решения

Разработанная библиотека реализована на стандарте C++23 и предоставляет механизмы контроля частоты запросов. Система поддерживает два фундаментальных подхода: отбрасывание превышающих лимит запросов (Rate Limiting) и их сглаживание через очереди ожидания (Traffic Shaping).

2.1. Полиморфные интерфейсы и слои архитектуры

Система спроектирована с использованием строгих контрактов на базе C++20 Concepts (RateLimiterLogic, Storage, PolyRateLimiter) и предоставляет два независимых виртуальных интерфейса:

- **IRateLimiter:** Интерфейс для классических лимитеров. Предоставляет методы проверки квоты `GetNumAvail()`, метод синхронного доступа `Access(key)` и метод проверки существования ключа `Exists(key)`.
- **IRateShaper:** Интерфейс для алгоритмов сглаживания трафика. Метод `AddRequest(key)` возвращает объект `std::optional<std::future<bool>>`, что позволяет вызывающему коду асинхронно ожидать своей очереди на выполнение.

Архитектура разделена на базовую логику ограничений (Algorithm), систему управления ключами и примитивами синхронизации (Storage) и связующий полиморфный слой (Wrapper).

2.2. Алгоритмы и механизм Traffic Shaping

Библиотека инкапсулирует логику лимитов в изолированных классах. Состояние каждого клиента обновляется в момент обращения с использованием монотонных часов `std::chrono::steady_clock`:

- **Token Bucket и Sliding Window Log:** Классические алгоритмы ограничения (Rate Limiting). Первый базируется на математической модели пополняемой емкости, второй использует очередь с приоритетом (`std::priority_queue`) для точного логгирования временных меток запросов.

- **Leaky Bucket (LeakyBucketShaper):** Реализует паттерн Traffic Shaping. Запросы, превышающие базовую пропускную способность, помещаются во внутреннюю очередь (`std::queue`). Выделенный фоновый поток (`queue_update_`) с заданной периодичностью извлекает запросы из очереди и сигнализирует об их готовности через механизм `std::promise`, разблокируя `std::future` на стороне клиента.

2.3. Управление временем жизни (TTL) и фоновая очистка

Для предотвращения утечек памяти при работе с неограниченным потоком уникальных ключей реализован механизм Time-To-Live (TTL). Логика отслеживания времени жизни интегрирована в алгоритмы с помощью паттерна CRTP (шаблонный класс-обертка `StoredData`), который добавляет методы `IsExpired()` и `Prolong()` без модификации базового кода лимитеров.

Очистка устаревших ключей (Eviction) в `MutexStorage` работает в двух режимах:

- **Фоновая очистка (Background Cleaner):** При инициализации хранилища запускается отдельный поток (`cleaner_thread_`), который периодически блокирует хранилище и удаляет истекшие записи (`DoCleanup()`).
- **Ленивая очистка (Lazy Cleanup):** Активируется только при конкурентных обращениях к хранилищу (с проверкой через атомарные флаги `LazyState`), что снижает накладные расходы на синхронизацию при слабой нагрузке.

2.4. Стратегии хранения ключей и потокобезопасность

Хранение состояния клиентов реализовано с учетом многопоточной конкуренции:

MutexStorage использует `std::unordered_map`, где каждый клиент защищен собственным мьютексом. Обеспечивает высокий параллелизм, но требует выделения примитива синхронизации на каждый ключ.

ShardedStorage группирует ключи в блоки (`DataNode`), защищая каждый шард одним мьютексом.

Для оптимизации пропускной способности в шардированном хранилище применяется борьба с эффектом ложного разделения кэш-линий (False Sharing). Выделение памяти под шарды производится через кастомный `AlignedAllocator` (с использованием `_aligned_malloc` или `std::aligned_alloc`), а объекты располагаются в памяти с выравниванием по границе 64 байт (`kCacheLineSize`), что исключает аппаратные конфликты кэша процессора между рабочими потоками.

3. Экспериментальная часть

4. Результаты работы